

Product Review and Rating Management System
Swiggy Instamart E-commerce Database Mangement System

1.Database Design

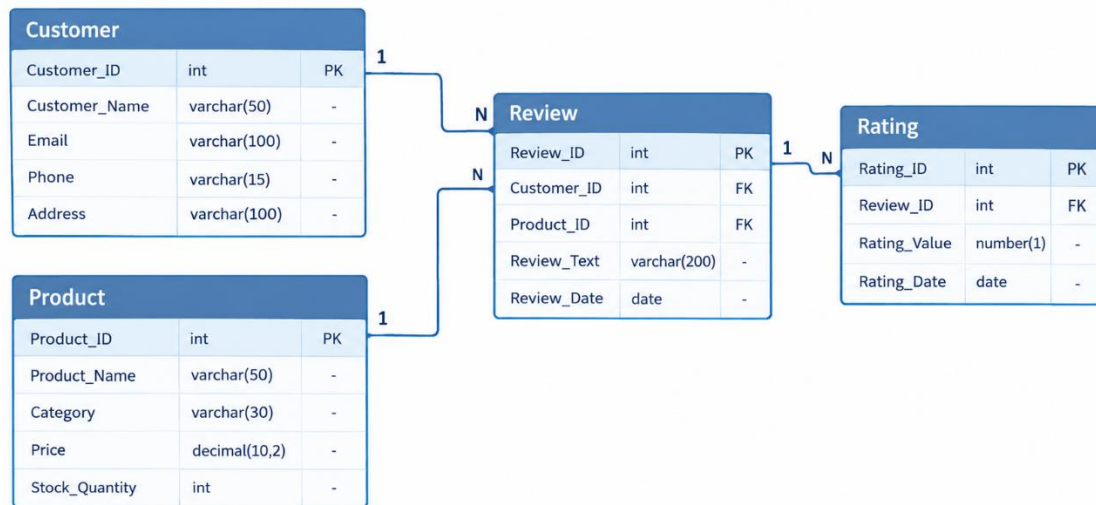
1.1 Review Table

Attribute	Data Type	Key Type
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Review_Text	VARCHAR2(200)	-
Review_Date	DATE	-

1.2 Rating Table

Attribute	Data Type	Key Type
Rating_ID	NUMBER	Primary Key
Review_ID	NUMBER	Foreign Key
Rating_Value	NUMBER(1)	-
Rating_Date	DATE	-

2. Table Relationship



3. Table Creation

3.1 Create Review Table

Create a table to store customer review for products.

```
SQL> CREATE TABLE Review (  
2     Review_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER,  
4     Product_ID NUMBER,  
5     Review_Text VARCHAR2(200),  
6     Review_Date DATE  
7 );
```

Table created.

3.2 Create Rating Table

Create a table to store product ratings

```
SQL> CREATE TABLE Rating (  
2     Rating_ID NUMBER PRIMARY KEY,  
3     Review_ID NUMBER,  
4     Rating_Value NUMBER(1),  
5     Rating_Date DATE  
6 );
```

Table created.

4.Review Data Insertion

Insert customer review records into the Review table.

```

SQL> INSERT INTO Review
  2 VALUES (1, 101, 1, 5, 'Fresh and good quality apples.',
  3          TO_DATE('01-10-2026', 'DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Review
  2 VALUES (2, 102, 2, 4, 'Bananas were fresh and tasty.',
  3          TO_DATE('02-10-2026', 'DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Review
  2 VALUES (3, 101, 3, 5, 'Fresh tomatoes and good quality.',
  3          TO_DATE('03-10-2026', 'DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Review
  2 VALUES (4, 102, 4, 4, 'Potatoes were fresh and good.',
  3          TO_DATE('04-10-2026', 'DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Review
  2 VALUES (5, 101, 5, 5, 'Orange juice tasted fresh and delicious.',
  3          TO_DATE('05-10-2026', 'DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Review
  2 VALUES (6, 102, 6, 4, 'Good taste and properly packed.',
  3          TO_DATE('06-10-2026', 'DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Review
  2 VALUES (7, 101, 7, 5, 'Crispy and tasty chips.',
  3          TO_DATE('07-10-2026', 'DD-MM-YYYY'));

1 row created.

```

5. Rating Data Insertion

```
SQL> INSERT INTO Rating VALUES (1, 1, 5, TO_DATE('01-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (2, 2, 4, TO_DATE('02-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (3, 3, 5, TO_DATE('03-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (4, 4, 4, TO_DATE('04-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (5, 5, 5, TO_DATE('05-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (6, 6, 4, TO_DATE('06-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (7, 7, 5, TO_DATE('07-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (8, 8, 4, TO_DATE('08-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (9, 9, 5, TO_DATE('09-10-2026', 'DD-MM-YYYY'));
1 row created.

SQL> INSERT INTO Rating VALUES (10, 10, 4, TO_DATE('10-10-2026', 'DD-MM-YYYY'));
1 row created.
```

6.Retrieve Product Review Details

Retrieve product wise customer review information.

```

SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     R.REVIEW_ID,
  5     R.CUSTOMER_ID,
  6     R.REVIEW_TEXT,
  7     R.REVIEW_DATE
  8 FROM PRODUCT P
  9 JOIN REVIEW R
 10     ON P.PRODUCT_ID = R.PRODUCT_ID;

```

PRODUCT_ID

PRODUCT_NAME

REVIEW_ID CUSTOMER_ID

REVIEW_TEXT

REVIEW_DA

```

      1
Milk      1          101

```

PRODUCT_ID

PRODUCT_NAME

REVIEW_ID CUSTOMER_ID

REVIEW_TEXT

REVIEW_DA

```

Fresh and good quality apples.
01-OCT-26

```

PRODUCT_ID

7. Calculate Average Product Ratings

Calculates the average rating of each product using aggregate functions.

```

SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     ROUND(AVG(RT.RATING_VALUE), 2) AS AVERAGE_RATING
  5 FROM PRODUCT P
  6 JOIN REVIEW R
  7   ON P.PRODUCT_ID = R.PRODUCT_ID
  8 JOIN RATING RT
  9   ON R.REVIEW_ID = RT.REVIEW_ID
 10 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11 ORDER BY P.PRODUCT_ID;

```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

	1	
Milk		5

	2	
Bread		4

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

	3	
Apple		5

	4	
Biscuits		

8. Identify Highly Rated Products

Identifies product with an average rating of 4 or above.

```

SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     ROUND(AVG(RT.RATING_VALUE), 2) AS AVERAGE_RATING
  5 FROM PRODUCT P
  6 JOIN REVIEW R
  7     ON P.PRODUCT_ID = R.PRODUCT_ID
  8 JOIN RATING RT
  9     ON R.REVIEW_ID = RT.REVIEW_ID
 10 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11 HAVING AVG(RT.RATING_VALUE) >= 4
 12 ORDER BY AVERAGE_RATING DESC;

```

```

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
          1
Milk          5

          3
Apple         5

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
          5
Fruit Juice   5

          4
Biscuits

```

9. product Rating Report

Generates a complete product wise rating summary using aggregate functions.

```

SQL> SELECT
  2     P.PRODUCT_ID,
  3     P.PRODUCT_NAME,
  4     COUNT(RT.RATING_ID) AS TOTAL_RATINGS,
  5     ROUND(AVG(RT.RATING_VALUE), 2) AS AVERAGE_RATING,
  6     MAX(RT.RATING_VALUE) AS HIGHEST_RATING,
  7     MIN(RT.RATING_VALUE) AS LOWEST_RATING
  8 FROM PRODUCT P
  9 JOIN REVIEW R
10     ON P.PRODUCT_ID = R.PRODUCT_ID
11 JOIN RATING RT
12     ON R.REVIEW_ID = RT.REVIEW_ID
13 GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
14 ORDER BY AVERAGE_RATING DESC;

```

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS AVERAGE_RATING HIGHEST_RATING LOWEST_RATING

	1			
Milk	1	5	5	5

	3			
Apple	1	5	5	5

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS AVERAGE_RATING HIGHEST_RATING LOWEST_RATING

	5			
Fruit Juice	1	5	5	5